

DNS_Local

LAB Setup Environment

Run the SEED VM>> open Firefox in that>> go to the seed labs and then click on the seed labs >> network security >> Select the DNS Lab >>> Download the .zip file.

Copy this zip to Documents (any other folder in VM), here I moved it to documents >> DNS>> there unzip the labsetup.zip file. From here right click and open the Terminal then we can directly move to Terminal.

Now, We need to Set up Containers, and then use the "dock ps" command to find out the ID of the container

```
seed@VM: ~/.../Labsetup
seed@VM: ~/.../Labsetup
[11/23/25] seed@VM: ~/.../Labsetup$ dockps
c50fc436fc6b  seed-attacker
7c19d0ee0f1e  attacker-ns-10.9.0.153
07f3b929c021  local-dns-server-10.9.0.53
dbd80a9eae4d  seed-router
d2df8d39b9a9  user-10.9.0.5
[11/23/25] seed@VM: ~/.../Labsetup$
```

Before performing the tasks we need to setup everything

Testing the DNS Setup

```
[11/23/25] seed@VM: ~/.../Labsetup$ docksh d2
root@d2df8d39b9a9:/# dig ns.attacker32.com

; <<>> DiG 9.16.1-Ubuntu <<>> ns.attacker32.com
;; global options: +cmd
;; Got answer:
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 1545
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
;; EDNS: version: 0, flags:; udp: 4096
;; COOKIE: 78cbf2785ef164310100000069229bebe707098b850d061a (good)
;; QUESTION SECTION:
;ns.attacker32.com.                IN      A

;; ANSWER SECTION:
ns.attacker32.com.                259200  IN      A      10.9.0.153

;; Query time: 0 msec
;; SERVER: 10.9.0.53#53(10.9.0.53)
;; WHEN: Sun Nov 23 05:30:19 UTC 2025
;; MSG SIZE rcvd: 90
```

Dig www.example.com

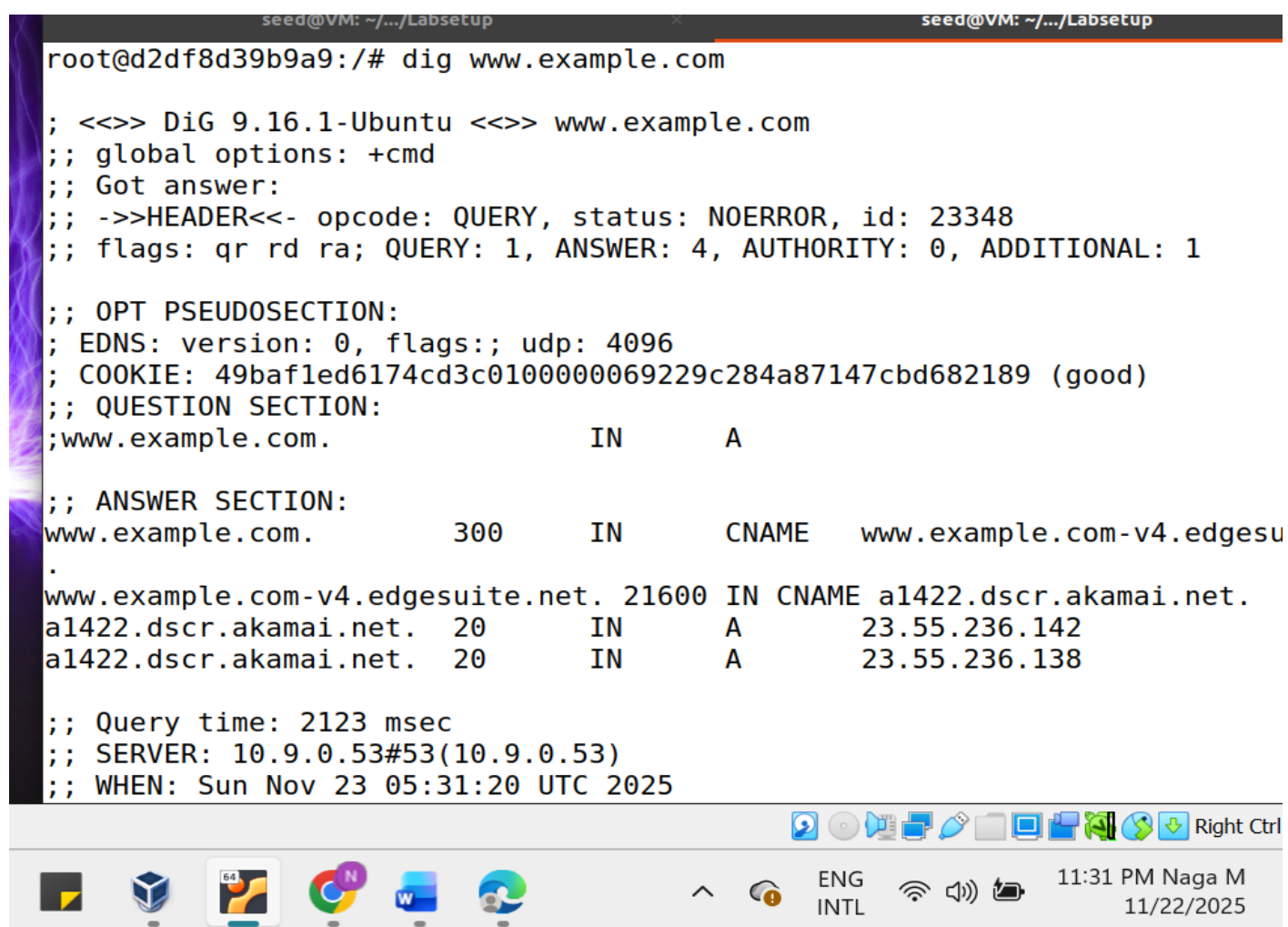
```
seed@VM: ~/.../Labsetup seed@VM: ~/.../Labsetup
root@d2df8d39b9a9:/# dig www.example.com

; <<>> DiG 9.16.1-Ubuntu <<>> www.example.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 23348
;; flags: qr rd ra; QUERY: 1, ANSWER: 4, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4096
; COOKIE: 49baf1ed6174cd3c0100000069229c284a87147cbd682189 (good)
;; QUESTION SECTION:
;www.example.com.                IN      A

;; ANSWER SECTION:
www.example.com.                 300     IN      CNAME   www.example.com-v4.edgesu
.
www.example.com-v4.edgesuite.net. 21600   IN      CNAME   a1422.dscr.akamai.net.
a1422.dscr.akamai.net.          20      IN      A        23.55.236.142
a1422.dscr.akamai.net.          20      IN      A        23.55.236.138

;; Query time: 2123 msec
;; SERVER: 10.9.0.53#53(10.9.0.53)
;; WHEN: Sun Nov 23 05:31:20 UTC 2025
```



Test querying attacker nameserver directly:

```

root@d2df8d39b9a9:/# dig @ns.attacker32.com www.example.com

; <<>> DiG 9.16.1-Ubuntu <<>> @ns.attacker32.com www.example.com
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 3349
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4096
; COOKIE: ed4e8d796f67169c0100000069229cc88b8d46b2a9f2de07 (good)
;; QUESTION SECTION:
;www.example.com.                IN      A

;; ANSWER SECTION:
www.example.com.                259200  IN      A      1.2.3.5

;; Query time: 0 msec
;; SERVER: 10.9.0.153#53(10.9.0.153)
;; WHEN: Sun Nov 23 05:34:00 UTC 2025
;; MSG SIZE rcvd: 88

root@d2df8d39b9a9:/#

```

If you want to force resolver behavior/remove local resolver caching issues on the user container you can check `/etc/resolv.conf`

```

root@d2df8d39b9a9:/# cat /etc/resolv.conf
nameserver 10.9.0.53
root@d2df8d39b9a9:/#

```

Local DNS Server. We run the BIND 9 DNS server program on the local DNS server.

```
[11/23/25]seed@VM:~/.../Labsetup$ docksh 07
root@07f3b929c021:/# rndc dumpdb -cache
root@07f3b929c021:/# cat /var/cache/bind/dump.db
```

```
;
; Start view _default
;
;
; Cache dump of view '_default' (cache _default)
;
; using a 604800 second stale ttl
$DATE 20251116054002
; authanswer
.          1122617 IN NS      a.root-servers.net.
          1122617 IN NS      b.root-servers.net.
          1122617 IN NS      c.root-servers.net.
          1122617 IN NS      d.root-servers.net.
          1122617 IN NS      e.root-servers.net.
          1122617 IN NS      f.root-servers.net.
          1122617 IN NS      g.root-servers.net.
          1122617 IN NS      h.root-servers.net.
          1122617 IN NS      i.root-servers.net.
          1122617 IN NS      j.root-servers.net.
          1122617 IN NS      k.root-servers.net.
          1122617 IN NS      l.root-servers.net.
```

Right Ctrl

11:41 PM Naga M
11/22/2025

```
root@07f3b929c021: /
seed@VM: ~/.../Labsetup x seed@VM: ~/.../Labsetup x root@07f3b929c021: / x
690677 DS 370 13 2 (
BE74359954660069D5C63D200C39F5603827
D7DD02B56F120EE9F3A86764247C )
; additional
690677 RRSIG DS 13 2 86400 (
20251130012137 20251123001137 46539 co
SdDmBMqLi4TgGwZz7QsIY8izRbbNujpp/Xc/
PGfMqDU/u3McMLX6ZQ8Bp3e7cKPBQQC8ldUw
qeI8nJb/JlKD+Q== )
; authanswer
www.example.com. 604577 CNAME www.example.com-v4.edgesuite.net.
; authanswer
604577 RRSIG CNAME 13 3 300 (
20251201111644 20251110104801 9776 exa
le.com.
1rRR10WnBP2I+tTqyJQhCNYXPLTUDAsCw/ZN
6xgCJ0Rt6+3wlAmcKmATadCCJXvUmmZ+b5+L
HJHERS3I5zdlyA== )
; glue
a0.org.afiliast-nst.info. 777077 A 199.19.56.1
; glue
777077 AAAA 2001:500:e::1
; glue
a2.org.afiliast-nst.info. 777077 A 199.249.112.1
```

Flush DNS cache (always do this **before** attacks):

```
; Dump complete
root@07f3b929c021:/# rndc flush
root@07f3b929c021:/# rndc dumpdb -cache
root@07f3b929c021:/# cat /var/cache/bind/dump.db
;
; Start view _default
;
; Cache dump of view '_default' (cache _default)
;
; using a 604800 second stale ttl
$DATE 20251116054315
;
; Address database dump
;
; [edns success/4096 timeout/1432 timeout/1232 timeout/512 timeout]
; [plain success/timeout]
;
;
; Unassociated entries
```

Check named configuration files:

cat /etc/bind/named.conf

```
;
; Dump complete
root@07f3b929c021:/# cat /etc/bind/named.conf
// This is the primary configuration file for the BIND DNS server named.
//
// Please read /usr/share/doc/bind9/README.Debian.gz for information on
// structure of BIND configuration files in Debian, *BEFORE* you customi
// this configuration file.
//
// If you are just adding zones, please do that in /etc/bind/named.conf.

include "/etc/bind/named.conf.options";
include "/etc/bind/named.conf.local";
include "/etc/bind/named.conf.default-zones";

zone "attacker32.com" {
    type forward;
    forwarders {
        10.9.0.153;
    };
};

root@07f3b929c021:/#
```

```
root@07f3b929c021: /
seed@VM: ~/.../Labsetup
seed@VM: ~/.../Labsetup
root@07f3b929c021: /
root@07f3b929c021:/# cat /etc/bind/named.conf.options
options {
    directory "/var/cache/bind";

    // If there is a firewall between you and nameservers you want
    // to talk to, you may need to fix the firewall to allow multiple
    // ports to talk.  See http://www.kb.cert.org/vuls/id/800113

    // If your ISP provided one or more IP addresses for stable
    // nameservers, you probably want to use them as forwarders.
    // Uncomment the following block, and insert the addresses replacing
    // the all-0's placeholder.

    // forwarders {
    //     0.0.0.0;
    // };

    //=====

    // If BIND logs error messages about the root key being expired,
    // you will need to update your keys.  See https://www.isc.org/bind-keys
    //=====

    // -----
    // Added/Modified for SEED labs
    // dnssec-validation auto;
    dnssec-validation no;
    dnssec-enable no;
    dump-file "/var/cache/bind/dump.db";
}
```

```
bash: systemctl: command not found
root@07f3b929c021:/# grep -R "attacker32.com" -n /etc/bind | cat /etc/bind/named.conf
/etc/bind/named.conf:13:zone "attacker32.com" {
root@07f3b929c021:/#
```

attacker-ns-10.9.0.153 (attacker's nameserver) — IP 10.9.0.153

Purpose: hosts attacker32.com and example.com zone files (the fake zones).

```
3: br-8eecaee5cc49: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether 02:42:9d:76:24:ef brd ff:ff:ff:ff:ff:ff
    inet 10.9.0.1/24 brd 10.9.0.255 scope global br-8eecaee5cc49
        valid_lft forever preferred_lft forever
    inet6 fe80::42:9dff:fe76:24ef/64 scope link
        valid_lft forever preferred_lft forever
4: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN
```

Task 1 — Directly spoofing response to the user

sniff a DNS query from the user machine and send fake DNS response directly to the **user** (not poisoning the DNS server)

before the attack once again try dig www.example.com in user

```
seed@VM: ~/.../Labsetup
seed@VM: ~/.../Labsetup x seed@VM: ~/.../Labsetup x seed@VM: ~/.../Labsetup x seed@VM: ~/.../Labsetup x
root@d51a2f58edc8:/# dig www.example.com

; <<>> DiG 9.16.1-Ubuntu <<>> www.example.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 36431
;; flags: qr rd ra; QUERY: 1, ANSWER: 4, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags;; udp: 4096
; COOKIE: 86d3431e20483e710100000069321d91b7ae381f34379306 (good)
;; QUESTION SECTION:
;www.example.com.                IN      A

;; ANSWER SECTION:
www.example.com.                292     IN      CNAME   www.example.com-v4.edgesuite.net
.
www.example.com-v4.edgesuite.net. 20861   IN      CNAME   a1422.dscr.akamai.net.
a1422.dscr.akamai.net.         12      IN      A        23.55.236.72
a1422.dscr.akamai.net.         12      IN      A        23.55.236.69

;; Query time: 0 msec
;; SERVER: 10.9.0.53#53(10.9.0.53)
;; WHEN: Thu Dec 04 23:47:29 UTC 2025
```

Open the Seed attacker machine and run the code

```
root@VM:/volumes# chmod a+x task1.py
root@VM:/volumes# python3 task1.py
[+] User query caught → TXID 35903
[+] Spoof sent to user!
[+] User query caught → TXID 23026
[+] Spoof sent to user!
```

And then try the dig www.example.com in user machine. After the attack, we can see the result as below

```
root@d51a2f58edc8:/# dig www.example.com

; <<>> DiG 9.16.1-Ubuntu <<>> www.example.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 35903
;; flags: qr aa rd; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 0
;; WARNING: recursion requested but not available

;; QUESTION SECTION:
;www.example.com.                IN      A

;; ANSWER SECTION:
www.example.com.                600     IN      A      1.2.3.4

;; Query time: 83 msec
;; SERVER: 10.9.0.53#53(10.9.0.53)
;; WHEN: Thu Dec 04 23:52:47 UTC 2025
;; MSG SIZE rcvd: 64

root@d51a2f58edc8:/#
```

From this result we can see that Spoofing was successful...

Task2 : DNS cache poisoning Attack – spoofing answers

Before start the task2, on the dns server use rndc flush and then try dig in user machine, check the result.

```
root@daef2a9f95df:/# dig www.example.com
```

```
; <<>> DiG 9.16.1-Ubuntu <<>> www.example.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 12035
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4096
; COOKIE: 3603279691c4dc6c01000000693238ad7f880c23f64f2eca (good)
;; QUESTION SECTION:
;www.example.com.                IN      A

;; ANSWER SECTION:
www.example.com.                 300     IN      A      10.9.0.1

;; Query time: 188 msec
;; SERVER: 10.9.0.53#53(10.9.0.53)
;; WHEN: Fri Dec 05 01:43:09 UTC 2025
;; MSG SIZE rcvd: 88
```

```
root@daef2a9f95df:/#
```

Right Ctrl



ENG INTL 7:44 PM Naga M 12/4/2025

```
root@VM:/volumes# python3 task2_poision.py
[*] Sniffing on interface: br-8eecaee5cc49
[*] Using filter: udp port 53 and src host 10.9.0.53
[*] Waiting for DNS queries from local DNS...
[DEBUG] DNS query .com. from 10.9.0.53 to 192.33.4.12
[DEBUG] DNS query . from 10.9.0.53 to 199.7.91.13
[DEBUG] DNS query .example.com. from 10.9.0.53 to 192.48.79.30
[+] Intercepted DNS query for .example.com.
[+] Sending spoofed reply to DNS server: .example.com. -> 10.9.0.1
[DEBUG] DNS query www.example.com. from 10.9.0.53 to 192.55.83.30
[+] Intercepted DNS query for www.example.com.
[+] Sending spoofed reply to DNS server: www.example.com. -> 10.9.0.1
[DEBUG] DNS query .com. from 10.9.0.53 to 192.5.5.241
[DEBUG] DNS query . from 10.9.0.53 to 199.7.83.42
[DEBUG] DNS query .example.com. from 10.9.0.53 to 192.33.14.30
[+] Intercepted DNS query for .example.com.
[+] Sending spoofed reply to DNS server: .example.com. -> 10.9.0.1
[DEBUG] DNS query www.example.com. from 10.9.0.53 to 192.5.6.30
[+] Intercepted DNS query for www.example.com.
[+] Sending spoofed reply to DNS server: www.example.com. -> 10.9.0.1
```

Task 2 is successfully completed. Your DNS cache poisoning attack worked perfectly. The fake IP 10.9.0.1 is now cached and will be served to all users until the cache expires or is manually flushed.

```
root@0bcbbee48ddb1:/# grep example /var/cache/bind/dump.db
_.example.com.        605093  A       10.9.0.1
www.example.com.      605093  A       10.9.0.1
root@0bcbbee48ddb1:/#
```

TASK3 :spoofing NS records (Authority section)

First in the attacker machine write the python script and run it and hen from the user I did the dig www.example.com command

```

root@VM:/volumes# python3 task3.py
[*] Sniffing on interface: br-8eecaee5cc49
[*] Using filter: udp port 53 and src host 10.9.0.53
[*] Waiting for DNS queries from local DNS...
[+] Intercepted DNS query for .example.com. from local DNS
[+] Sending spoofed NS reply: example.com → ns.attacker32.com. (10.9.0.153)
[+] Intercepted DNS query for www.example.com. from local DNS
[+] Sending spoofed NS reply: example.com → ns.attacker32.com. (10.9.0.153)
[+] Intercepted DNS query for www.example.com. from local DNS
[+] Sending spoofed NS reply: example.com → ns.attacker32.com. (10.9.0.153)
[+] Intercepted DNS query for .example.com-v4.edgesuite.net. from local DNS
[+] Sending spoofed NS reply: example.com → ns.attacker32.com. (10.9.0.153)
[+] Intercepted DNS query for www.example.com-v4.edgesuite.net. from local DNS
[+] Sending spoofed NS reply: example.com → ns.attacker32.com. (10.9.0.153)
[+] Intercepted DNS query for .example.com. from local DNS
[+] Sending spoofed NS reply: example.com → ns.attacker32.com. (10.9.0.153)
[+] Intercepted DNS query for www.example.com. from local DNS
[+] Sending spoofed NS reply: example.com → ns.attacker32.com. (10.9.0.153)
[+] Intercepted DNS query for .example.com. from local DNS
[+] Sending spoofed NS reply: example.com → ns.attacker32.com. (10.9.0.153)

```

```

root@daef2a9f95df:/# dig www.example.com

; <<>> DiG 9.16.1-Ubuntu <<>> www.example.com
;; global options: +cmd
;; Got answer:
;; ->HEADER<<- opcode: QUERY, status: NOERROR, id: 8095
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4096
; COOKIE: 7442591c23d715230100000069322e3f30db49fe4f1aecde (good)
;; QUESTION SECTION:
;www.example.com.                IN      A

;; ANSWER SECTION:
www.example.com.                259200  IN      A      1.2.3.5

;; Query time: 251 msec
;; SERVER: 10.9.0.53#53(10.9.0.53)
;; WHEN: Fri Dec 05 00:58:39 UTC 2025
;; MSG SIZE rcvd: 88

root@daef2a9f95df:/#

```

```
root@0bcbbee48ddb1:/# grep example /var/cache/bind/dump.db
example.com.        605069  NS      ns.attacker32.com.
www.example.com.    863969  A       1.2.3.5
root@0bcbbee48ddb1:/#
```

Both the spoofed A record (www.example.com = 1.2.3.5) and NS record (example.com → ns.attacker32.com) were successfully cached. **Explanation:** Same-domain NS poisoning bypasses bailiwick checks, allowing attackers to redirect all subdomain queries to their nameserver.

Task4: : spoofing NS records for another domain

Run the script in attacker machine and dig in user

```

root@VM:/volumes# python3 task4_sp.py
[*] Starting Task 4: Cross-domain NS Poisoning
[*] Sniffing on: br-8eecaee5cc49
[*] Filter: udp port 53 and src host 10.9.0.53
[*] Will poison google.com when example.com is queried
[*] Waiting for DNS queries...
[+] Intercepted DNS query for _.example.com.
    Query was for: example.com
    Will poison: google.com
[+] Sending cross-domain spoofed response:
    Query: _.example.com.
    Poisoning: google.com → ns.attacker32.com.
    Nameserver IP: 10.9.0.153
[+] Intercepted DNS query for www.example.com.
    Query was for: example.com
    Will poison: google.com
[+] Sending cross-domain spoofed response:
    Query: www.example.com.
    Poisoning: google.com → ns.attacker32.com.
    Nameserver IP: 10.9.0.153

```

```

; <<>> DiG 9.16.1-Ubuntu <<>> www.example.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 6679
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4096
; COOKIE: 8de9ff4fa53338e10100000069323f0d48a1ae82e5f35f98 (good)
;; QUESTION SECTION:
;www.example.com.                IN      A

;; ANSWER SECTION:
www.example.com.                259200  IN      A      1.2.3.4

;; Query time: 404 msec
;; SERVER: 10.9.0.53#53(10.9.0.53)
;; WHEN: Fri Dec 05 02:10:21 UTC 2025
;; MSG SIZE rcvd: 88

root@daef2a9f95df:/#

```

See the result in dns,

```

root@0bcbee48ddb1:/# grep example /var/cache/bind/dump.db
_.example.com.                863992  A      1.2.3.4
www.example.com.              863992  A      1.2.3.4
root@0bcbee48ddb1:/#

```

They show that modern DNS servers have protections against cross-domain cache poisoning. Only example.com A records were cached (1.2.3.4), but the cross-domain google.com NS record was rejected. **Explanation:** DNS bailiwick checking prevents caching of unrelated authority records, blocking cross-domain poisoning while allowing same-domain spoofing.

Task 5: Spoofing Records in the Additional Section

Run the task5 script in attacker machine

```
root@VM:/volumes# python3 task5_sp.py
[*] Sniffing on br-8eecaee5cc49
[*] Filter: udp and src host 10.9.0.53 and dst port 53
[*] Waiting for www.example.com queries from local DNS server...
[+] Intercepted DNS query for www.example.com.
[+] Sending spoofed DNS reply with:
    Answer: www.example.com → 10.9.0.1
    Authority: example.com NS ns.attacker32.com
    Additional: 3 A records (attacker32, example.net, facebook)
[+] Intercepted DNS query for www.example.com.
[+] Sending spoofed DNS reply with:
    Answer: www.example.com → 10.9.0.1
    Authority: example.com NS ns.attacker32.com
    Additional: 3 A records (attacker32, example.net, facebook)
```

Dig in user machine,

```
root@daef2a9f95df:/# dig www.example.com

; <<>> DiG 9.16.1-Ubuntu <<>> www.example.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 60035
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4096
; COOKIE: 1070fc38fa2884ec01000000693235041d372a6985411175 (good)
;; QUESTION SECTION:
;www.example.com.                IN      A

;; ANSWER SECTION:
www.example.com.                300     IN      A      10.9.0.1

;; Query time: 524 msec
;; SERVER: 10.9.0.53#53(10.9.0.53)
;; WHEN: Fri Dec 05 01:27:32 UTC 2025
;; MSG SIZE rcvd: 88

root@daef2a9f95df:/#
```

Observe the cache in dns server

```
root@0bcbbee48ddb1:/# grep example /var/cache/bind/dump.db
example.com.          777587 NS      ns.attacker32.com.
www.example.com.      863987 A       10.9.0.1
root@0bcbbee48ddb1:/#
```

Answer based on our results:

1. Cached entries:

- www.example.com A 10.9.0.1 - **Answer section**, directly responds to the query
- example.com NS ns.attacker32.com - **Authority section**, legitimate NS record

2. Not cached entries (all from Additional section):

- ns.example.net A 5.6.7.8 - NS not mentioned in Authority
- www.facebook.com A 3.4.5.6 - Completely unrelated.

So we can come up with - DNS servers are selective about what they cache,

Additional section records often don't get cached, This is a security feature against certain types of cache poisoning